

Solucion/Memoria tarea 14

Para que podamos crear un video primero necesitamos tener un guión, un lugar de donde sacar el texto que se va a decir, para ello está este método donde dentro de la carpeta docs existe un archivo llamado input.pdf.

 input.pdf

Este contiene todo el guion (texto) de nuestro video.

Con este método leemos el texto y lo dividimos en partes (diferentes escenas) para que la IA lo pueda procesar.

Para ello hemos tenido que limpiar ese texto que nos venía y dividirlo por frases.

```
def leer_pdf_y_dividir(ruta_pdf):  
    Lee un archivo PDF, extrae su texto y lo divide en escenas.  
  
    Args:  
        ruta_pdf (str): La ruta al archivo PDF del guion.  
  
    Returns:  
        list: Una lista de strings, donde cada string es una escena (un grupo de frases).  
    """  
    # Usamos pypdf para leer el archivo  
    reader = PdfReader(ruta_pdf)  
  
    texto_completo = ""  
  
    # Iteramos por cada página para extraer el texto crudo  
    for page in reader.pages:  
        texto_completo += page.extract_text()  
  
    # Convertimos los saltos de línea en espacios  
    texto_limpio = texto_completo.replace('\n', ' '  
  
    # Dividimos el texto en frases  
    frases = [f.strip() + "." for f in texto_limpio.split('.') if len(f) > 10]  
  
    # Agrupamos las frases en bloques para tener más contexto.  
    escenas = []  
    chunk_size = 2  
    for i in range(0, len(frases), chunk_size):  
        bloque = " ".join(frases[i:i+chunk_size])  
        if bloque:  
            escenas.append(bloque)  
  
    # Retornamos solo las primeras 4 escenas para hacer un tráiler  
    return escenas[:4]
```

Cargamos el modelo y utilizamos TinyLlama como modelo de IA.

```
# Cargar el modelo  
pipe_llm = pipeline("text-generation", model="TinyLlama/TinyLlama-1.1B-Chat-v1.0", torch_dtype=torch_dtype, device=device)
```

Convertimos el texto en video, para ello le ponemos al modelo una temperatura de 0.7 para que sea ligeramente imaginativo y aporte algo fuera del guión.

```
def generar_prompt_base(texto_escena):
    """
    Utiliza el LLM para convertir el texto en video

    Args:
        texto_escena (str): El texto

    Returns:
        str: Una descripción visual optimizada para generadores de imágenes.
    """
    # Estructura de chat específica requerida por el modelo TinyLlama (<|user|> ... <|assistant|>)
    prompt = f"<|user|>\nDescribe the visual action in this scene for a movie shot in 15 words:\n{texto_escena}\n<|assistant|>"

    # Generamos la respuesta con parámetros para dar creatividad
    outputs = pipe_llm(prompt, max_new_tokens=60, do_sample=True, temperature=0.7)

    # Limpiamos la salida para quedarnos solo con la respuesta del asistente
    return outputs[0]['generated_text'].split("<|assistant|>")[-1].strip()
```

Con este método le añadimos al prompt vestuario de los protagonistas de nuestra historia. Lo añade al texto que se le pasará a la IA

```
def inyectar_vestuario(texto_original, prompt_base):
    """
    Enriquece el prompt con detalles estilísticos y específicos de los personajes.
    """
    texto_lower = texto_original.lower()
    prompt_mejorado = prompt_base

    # Detectamos palabras clave en el guion original para añadir descripciones visuales precisas.
    if 'atom' in texto_lower:
        prompt_mejorado += ", (rusted grey fighting robot:1.5), glowing blue eyes, mechanical face, boxing stance, battle damage"
    elif 'zeus' in texto_lower:
        prompt_mejorado += ", (massive shiny black robot:1.5), green neon lights, futuristic armor, menacing look"
    elif 'charlie' in texto_lower:
        prompt_mejorado += ", (man with short beard and leather jacket:1.2), looking at robot"
    else:
        # Si no se menciona un personaje esp Literal[, robots boxing in a dirty arena, metal textures']
        prompt_mejorado += ", robots boxing in a dirty arena, metal textures"

    # Keywords técnicas para forzar al modelo a generar imágenes de alta calidad tipo cine.
    # Estrategia Close-Up para evitar caras deformes si están muy lejos.
    prompt_mejorado += ", close-up shot, 8k, cinematic lighting, photorealistic"

    return prompt_mejorado
```

Pasamos de texto a imagen

```
# Stable Diffusion se encarga de convertir el texto a imagen. Usamos "Realistic Vision" para estilo fotorealista.
pipe_img = StableDiffusionPipeline.from_pretrained("SG161222/Realistic_Vision_V5.1_noVAE", torch_dtype=torch_dtype)
pipe_img = pipe_img.to(device)
```

Generamos una imagen (escena) y la guardamos, estas imágenes posteriormente se utilizarán para el video.

```
def generar_imagen(prompt_final, nombre_archivo):
    """
    Genera una imagen a partir del prompt y la guarda en disco.
    """
    image = pipe_img(
        prompt=prompt_final,
        negative_prompt="cartoon, drawing, anime, deformed, bad anatomy, disfigured, human skin on robot",
        num_inference_steps=40, # Más pasos = Mayor calidad (pero más tiempo).
        guidance_scale=7.0,     # Qué tanto debe obedecer al prompt
        height=512, width=512   # Resolución cuadrada estándar
    ).images[0]
    image.save(nombre_archivo)
```

Generamos el audio del video

```
async def generar_audio(texto_ingles, nombre_archivo):
    """
    Traduce el guion al español y genera una locución con voz
    """
    # Traducimos al español
    traductor = GoogleTranslator(source='auto', target='es')
    texto_espanol = traductor.translate(texto_ingles)

    # Generamos voz neural con edge-tts (servicio de Microsoft) para una voz natural.
    communicate = edge_tts.Communicate(texto_espanol, 'es-ES-AlvaroNeural')
    await communicate.save(nombre_archivo)
```

Juntamos las imágenes del método generar_imagen y el audio del método generar_audio y creamos el video

```
def montar_video(datos_escenas, salida="trailer_acero_puro.mp4"):
    """
    Junta las imágenes y audios generados en un archivo de video final MP4.
    """
    clips = []
    for escena in datos_escenas:

        # Cargamos el archivo de audio generado
        audio = AudioFileClip(escena['audio'])

        # Creamos un clip de imagen estática
        # .with_duration: Hacemos que la imagen dure lo mismo que el audio.
        # .with_audio: Pegamos el audio a la imagen.
        clip = ImageClip(escena['imagen']).with_duration(audio.duration).with_audio(audio)

        # Efecto de transición
        clip = clip.with_effects([vfx.CrossFadeIn(0.5)])

        clips.append(clip)

    # Concatenamos todos los clips en secuencia
    video_final = concatenate_videoclips(clips, method="compose")

    # Renderizamos y guardamos el archivo final. 24 fps es estándar para cine.
    video_final.write_videofile(salida, fps=24)
```

En la función main vamos llamando a los diferentes métodos anteriores para crear el audio, imagenes y posteriormente el vídeo

```
async def main():  
    # Leemos el guion y lo preparamos  
    escenas_texto = leer_pdf_y_dividir("docs\input.pdf")  
    datos_para_montaje = []  
  
    # Procesamos cada escena  
    for i, texto in enumerate(escenas_texto):  
        print(f"Procesando escena {i+1}...")  
  
        # El LLM imagina la escena  
        prompt_base = generar_prompt_base(texto)  
  
        # Ajustamos el prompt con el estilo de nuestra película  
        prompt_final = inyectar_vestuario(texto, prompt_base)  
  
        print(f"Prompt Generado: {prompt_final}")  
  
        archivo_img = f"frame_{i}.png"  
        archivo_audio = f"audio_{i}.mp3"  
  
        # Generamos la imagen  
        generar_imagen(prompt_final, archivo_img)  
  
        # Generamos el audio  
        await generar_audio(texto, archivo_audio)  
  
        # Guardamos la referencia de los archivos creados para el montaje  
        datos_para_montaje.append({  
            'imagen': archivo_img,  
            'audio': archivo_audio  
        })  
  
    # Montamos el video  
    montar_video(datos_para_montaje)  
    print(";Corte! Película terminada.")
```